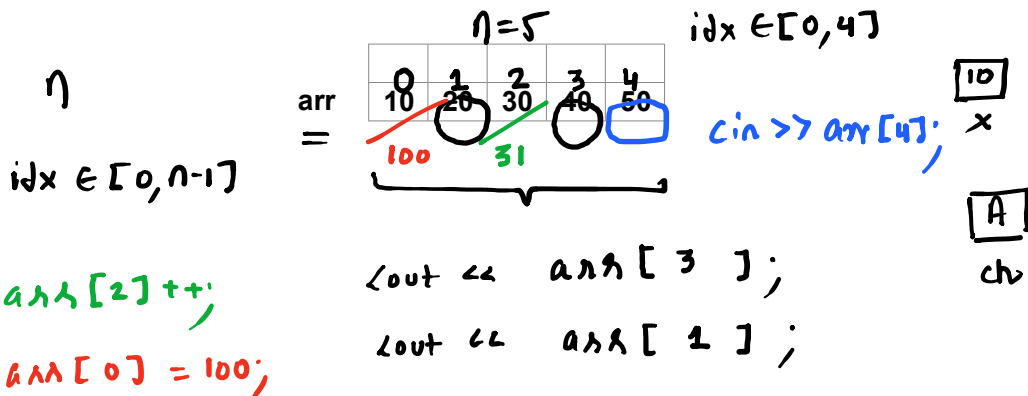


`<out << arr[6];` // undefined // avoid this

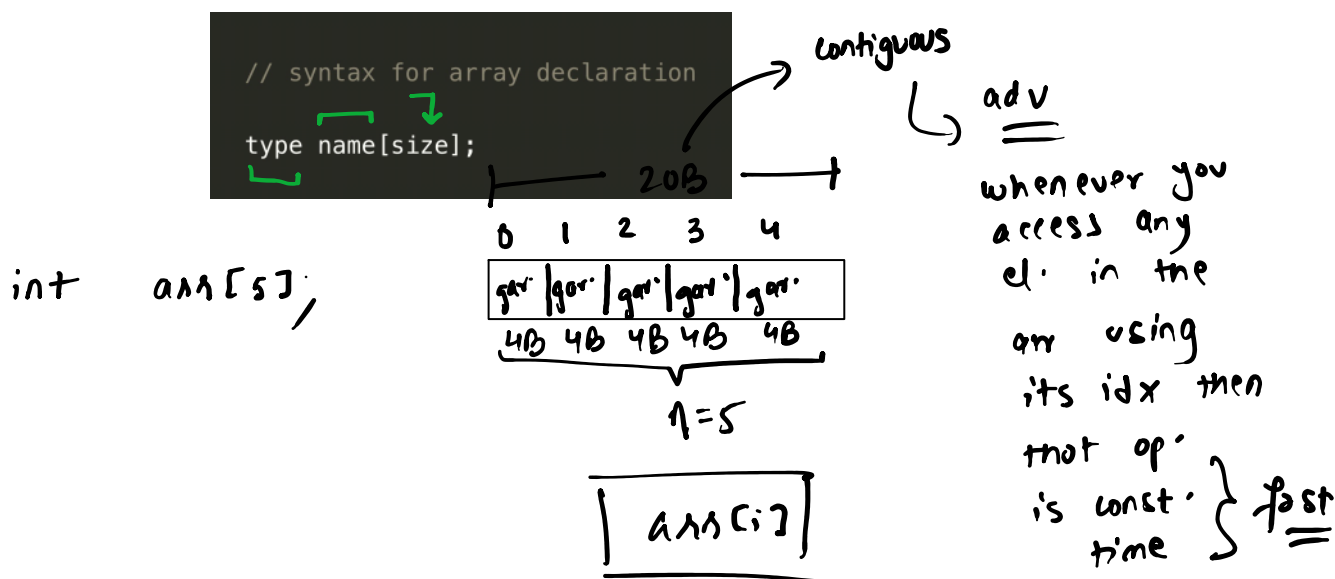
Introduction to Arrays

An array is a **linear** data structure, referred by a single name, which is used to store a **sequence** of values of the **same type**.

`arr[i]` $0 \leq i \leq n-1$



How to **declare** an array in C++ ?

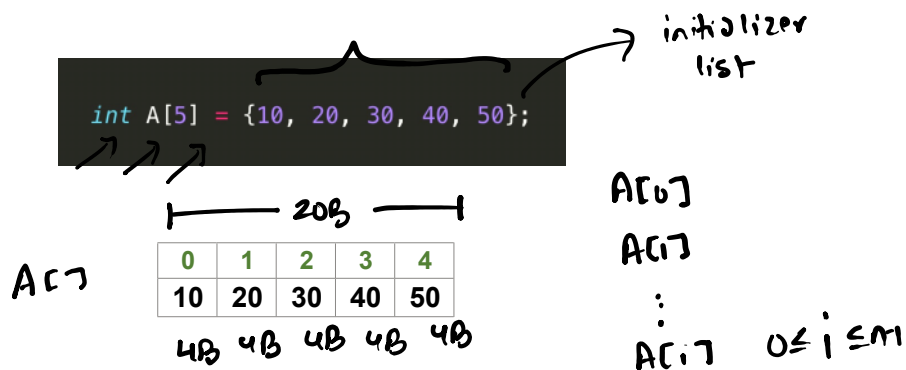


Since arrays are allocated memory in a **contiguous** fashion, we can randomly access an element in an array using its index in **constant time** i.e. **$O(1)$** time.

Array Initialization

Array Initialization

By default, when we declare an array in local-scope, it contains **garbage** value.



During the initialization of an array, specifying the size of the array is **optional**.

During initialization, the size of the **initializer list** must not exceed the array size.

`int A[5] = {10, 20, 30, 40, 50, 60};` error

What if the size of initializer list is less than array size ?

`int A[5] = {10, 20, 30};` Remaining slots will be filled with zero

↳ Reading values into array

$n \leq 100$

`int arr[100];` // based on constraints

`int n;`
`cin >> n;`

`cin >> arr[0];`
`cin >> arr[1];`
`... arr[n];`

Diagram illustrating array reading constraints:

The diagram shows an array of size 100. The first 5 elements (indices 0 to 4) are labeled "n" and "waste" (referring to the unused space). The remaining elements (indices 5 to 99) are labeled "waste". A bracket below the first 5 elements is labeled "n=5". Below the diagram, a loop is shown: `for (int i=0; i<n; i++) { cin >> arr[i]; }`

```
cin >> arr[0];  
cin >> arr[1];  
cin >> arr[2];  
cin >> arr[n];
```

```
for (int i=0; i<n; i++) {  
    cin >> arr[i];  
}
```

First Occurrence

First Occurrence

Given an array of **N** integers, and an integer **T**, design an algorithm to find the **index** of the **first occurrence** of T in the given array.

note : output **-1** if T is not present in the given array.

Example

Input : N = 5 ; T = 20

0	1	2	3	4
10	20	30	10	20

Output : 1

T = 100
o/p : -1

=

[HW] Last Occurrence

Last Occurrence

Given an array of **N** integers, and an integer **T**, design an algorithm to find the **index** of the **last occurrence** of T in the given array.

note : output **-1** if T is not present in the given array.

Example

Input : N = 5 ; T = 10

0	1	2	3	4
10	20	30	10	20

Output : 3

All Occurrences

All Occurrences

Given an array of **N** integers, and an integer **T**, design an algorithm to find the **indices** of **all occurrence** of T in the given array.

note : output **-1** if T is not present in the given array.

Example

Input : N = 5 ; T = 20

0	1	2	3	4
10	20	30	10	20

Output : 1, 4



Reverse Array

Reverse Array

Given an array of N integers, design an algorithm to reverse it.

Example

Input

0	1	2	3	4	5	6
10	20	30	40	50	60	70

Output

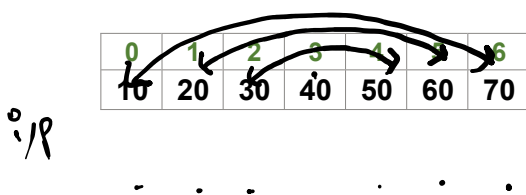
0	1	2	3	4	5	6
70	60	50	40	30	20	10

...

int x=10;
int y=20;

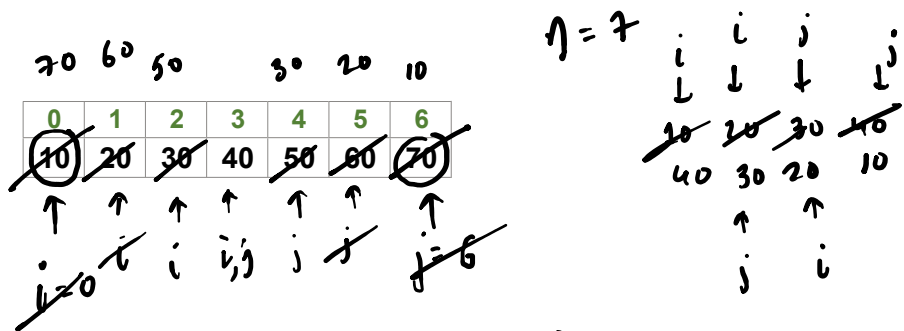
swap (x , y);

cout << x; // 20
cout << y; // 10



o/p 70 60 50 40 30 20 10

2 ptr Appr



$i < j$

swap (arr[i], arr[j])
i++;
j--;

int x = 10;
int y = 20;
int temp;

20
10
x
y
temp

int a = 5;
int b = 10;

a
5
b
10
5

a = a + b;
b = a - b;

b = a - b

```
int temp;
temp = x;
x = y;
y = temp;
```

10
temp

$a = a + b;$
 $\Rightarrow b = a - b;$
 $a = a - b;$

$b = \frac{a}{(a+b)}$
 $a = \frac{a}{a+b} - \frac{b}{a}$

$x = x \wedge y$
 $y = x \wedge y$
 $x = x \wedge y$

$y = (x \wedge y) \wedge y$
 $x = (x \wedge y) \wedge x$

Reverse in Range

Given an array of N integers, and two non-negative integers i and j where $0 \leq i < j \leq N-1$ design an algorithm to reverse the subarray that starts at the i^{th} index and ends at the j^{th} index.

Example

Input
 $i = 2, j = 5$

0	1	2	3	4	5	6
10	20	30	40	50	60	70

Output

0	1	2	3	4	5	6
10	20	60	50	40	30	70

0	1	2	3	4	5	6
10	20	30	40	50	60	70

Rotate Array

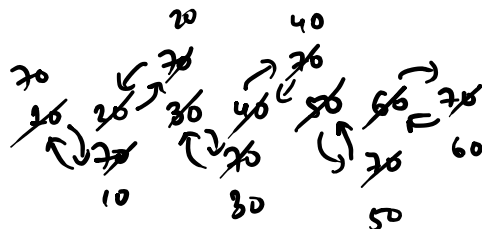
Given an array of **N** integers, design an algorithm to **rotate** the array once in the **clockwise** direction.

Example

Input						
0	1	2	3	4	5	6
10	20	30	40	50	60	70

0	1	2	3	4	5	6
70	10	20	30	40	50	60

0	1	2	3	4	5	6
10	20	30	40	50	60	70



```

swap(arr[6], arr[5])
swap(arr[5], arr[4])
swap(arr[4], arr[3])
swap(arr[3], arr[2])
swap(arr[2], arr[1])
⇒ swap(arr[1], arr[0])

```

```

⇒ swap(arr[i], arr[i-1])

```

init ⇒ $i = n-1$ to 1

K-Rotate Array

Given an array of **N** integers, and a positive integer **K** design an algorithm to **K-rotate** the array in the clockwise direction.

Example

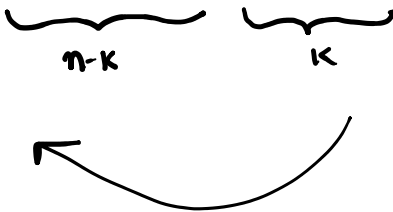
Input						
N = 7 ; K = 3						
0	1	2	3	4	5	6
10	20	30	40	50	60	70

Output

0	1	2	3	4	5	6
50	60	70	10	20	30	40

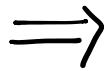
N = 7 ; K = 3

0	1	2	3	4	5	6
10	20	30	40	50	60	70



$$\frac{n}{2} + \frac{k}{2} + \frac{n-k}{2} = \frac{n}{2} + \frac{k}{2} + \frac{n-k}{2} = n \cdot c \therefore O(n)$$

Reverse



70 60 50 40 30 20 10

rev arr[0...k-1]

rev arr[k...n-1]

⇒ 50 60 70 10 20 30 40

N = 7 ; K = 3

0	1	2	3	4	5	6
10	20	30	40	50	60	70

n-7

10 20 30 0 rotations

30 10 20 k=1 rot.

20 30 10 k=2 rot

10 20 30 k=3 rot

30 10 20 k=4 rot

20 30 10 k=5 rot

$k-1 \cdot n$

Three Largest Elements

Given an array of **N** integers, design an algorithm to find the **three largest** elements present inside the given array

Example

Input

0	1	2	3	4	5
20	5	0	25	15	10

Solve this in $O(n)$ time.

Output

15	20	25
----	----	----

• •

Maximum Product of a Triplet

Given an array of **N** integers, design an algorithm to find the **maximum product** of a triplet present inside the given array.

Example

Input

0	1	2	3	4
20	5	25	15	10

Output : 7500

$$25 \times 20 \times 15$$

Input

0	1	2	3	4
-10	-20	1	2	3

Output : 600